IMPLEMENTATION AUDIT

Retail Agent: CoALA + Fan & Lan (2026)

Code review, simulator assessment, target runtime design, and LangChain4j portability

Review basis	Details
Project examined	Uploaded retail-agent.zip: Java 21, Spring Boot 4.1.0, Spring AI 2.0.1, LangGraph4j 1.8.25.
Paper baseline	Sumers et al., Cognitive Architectures for Language Agents (CoALA); Fan & Lan, From Cognitive Architectures to Language Agents: A Mechanism-Level Review of Lineage, Convergence, and Migration Gaps (2026).
Review date	4 November 2026.
Build status	The user reports compilation succeeds after identifying a single serialization defect. This review inspected source and tests; it did not rerun Maven.
Review purpose	Assess what the project demonstrates, what it does not, how faithful the simulator is, and how to strengthen it without misattributing paper mechanisms.

Executive verdict A useful deterministic LangGraph4j teaching skeleton, but not yet a complete CoALA implementation and not yet a demonstration of Fan & Lan's residual control mechanisms. Its outer loop is iterative; its planning sub-process is not. The simulator is suitable for deterministic plumbing and fault-path tests, but currently overstates semantic planning and verification.

This report distinguishes paper statements, observations from the uploaded code, and engineering recommendations. "Aligned with" means the code has an observable mechanism corresponding to the paper—not that it uses similar labels or prompts.

1. Executive findings

Area	Current implementation	Assessment
CoALA architecture	Working state, episodic notes, catalog vector store, internal reasoning/retrieval, whitelisted grounding tools, repeated decision cycles.	Partial but recognizable CoALA mapping.
CoALA planning	Exactly one propose → evaluate → select pass per outer cycle. No planning-time reject-and-repropose edge or deliberate interleaving.	Main conceptual defect. The outer action cycle loops, but the planning sub-process is rigid.
Execution and feedback	One chosen tool is executed; textual observation is appended; reflection decides whether to repeat.	Good skeleton; observations and outcomes need typed semantics.
Learning	At completion, a short string is appended to process-local episodic memory.	A valid small CoALA-style deferred-learning choice, but shallow and not durable.
Fan & Lan B1/B2/B3/B4/B6	No activation/latency/value/cost coupling, typed impasse substate, bounded workspace competition/broadcast, intention governor, or propagated calibrated uncertainty.	Not implemented in the uploaded code.

Area	Current implementation	Assessment
Fan & Lan B5	No GraSP-like typed skill DAG or local repair protocol.	B5 is not an unresolved residual hypothesis in the paper; it is the closed-case baseline to compare against.
LLM simulation	Scripted output rules, recording/replay, and prompt-level fault injection.	Strong test infrastructure concept; weak simulation of actual model reasoning.
Portability	Agent behavior is tightly coupled to Spring AI ChatModel and LangGraph4j AgentState/StateGraph.	Portable concepts, not yet portable implementation. Introduce framework-neutral core contracts first.

The two core papers answer different questions. CoALA organizes an agent around information storage, action space, and a repeated decision-making procedure. Fan & Lan provide a mechanism-level way to examine what state a mechanism consumes, where control resides, what triggers it, what persists, how failures are handled, how learning changes future behavior, and which resources/governance constraints apply. The latter does not turn every proposed bundle into a universal requirement. [R1][R2]

2. The immediate build issue

The uploaded Proposal.java record is not Serializable, while RetailState stores List<Proposal> under PROPOSALS and a Proposal under SELECTED. Given the user-reported LangGraph4j state-copy behavior, that directly explains the serialization exceptions. The record components are String and primitive double values, so the smallest source correction is exactly one declaration change:

```
public record Proposal(String tool, String args, String rationale,
                       double confidence, double score) implements java.io.Serializable {
```

No custom serialization logic is needed for those record components. Keep the state payload restricted to simple values and serializable domain records: strings, numbers, booleans, lists/maps of serializable values, and immutable DTOs. Do not put framework services, vector-store Documents, ChatResponse objects, open streams, lambdas, or Throwable instances into graph state.

> **Regression protection** Add one focused test that serializes/deserializes a Proposal and a representative RetailState snapshot containing both PROPOSALS and SELECTED. The user's existing passing tests demonstrate other behaviors; they do not, by themselves, lock this serialization contract.

The POM pins LangGraph4j 1.8.25. The project build should remain version-pinned while fixing and validating this defect; evaluate a LangGraph4j upgrade in a separate change so API or serializer changes do not get mixed with the model fix. The official release page shows 1.9.8 was released on 3 November 2026, after the project's pinned 1.8.25 line. [R3]

3. What the current graph actually does

```
START → retrieve → propose → evaluate → select → execute → reflect
                                ↑                    |
                                └──── retrieve ←─────┘
select → clarify/abort → answer → END
select/reflect → finish → learn → answer → END
```

The trace proves a real outer action loop. After a grounding action, reflect can route back to retrieve, and the next decision cycle sees the new observations and refreshed memory. The current ScriptedAgentTest explicitly asserts this two-cycle trace. Therefore, saying “there is no iteration at all” would be inaccurate.

The defect is inside each planning pass. RetailAgent.buildGraph wires propose → evaluate → select as a fixed chain. select chooses the maximum score, clarifies only under a narrow first-turn threshold condition, blocks finish when no

observation exists, or routes onward. There is no edge from a rejected selection back to proposal, no planning-round counter, and no retrieval/simulation branch within the planning process. reflect is post-execution; it cannot substitute for iterative planning before action.

Question	Code evidence	Implication
Can planning reject all candidates and ask for new ones?	No. Empty candidate list aborts; otherwise select takes argmax.	Missing CoALA selection→proposal loop.
Can retrieval and reasoning interleave during planning?	retrieve occurs before propose and again only after execution/reflect.	No planning-time retrieve → revise → evaluate cycle.
Can it simulate a candidate before acting?	No explicit rollout/simulator or hypothetical state transition.	No multi-step foresight or simulated feedback.
Are proposals multi-step plans?	Proposal is one tool name plus free-text args/rationale.	It is an action candidate, not a plan/trajectory.
Does reflection reconsider the method?	It returns summary and a stop/continue decision.	No B4 authority to switch method while preserving goal.
Can one execution be an internal learning action?	learn is a terminal graph node, not a selectable proposal.	Simplified but still consistent with CoALA's example that learning may be deferred to interaction end.

CoALA §4.6 explicitly allows reasoning and retrieval to propose, evaluate, and select actions flexibly; the sub-stages may interleave or iterate, build multi-step simulations, and refine candidate solutions. Selection may reject all candidates and loop back to proposal. Execution may apply either external grounding or an internal learning action. [R1]

4. CoALA alignment audit

CoALA dimension	Where it appears	Gap / correction
Working memory	RetailState carries query, customer, iteration, memory text, observations, proposals, selected action, route, failures, summary, answer, trace.	Mostly a working-memory analogue. It is created per ask and is not checkpointed across a suspended run or process restart.
Episodic long-term memory	EpisodicMemory stores previous query/outcome strings and retrieve puts recent notes into the prompt.	Process-local, unbounded, concurrency-unsafe ArrayList values inside ConcurrentHashMap; no expiry, schema, provenance, or durable persistence.
Semantic long-term memory	ProductCatalog is the system of record; a VectorStore indexes product descriptions.	Appropriate read-only semantic store shape, but test embedding is a token-hash baseline, not a semantic embedding model.
Procedural long-term memory	Java code, prompts, tools, and model weights implicitly embody procedures.	The project does not model a writable skill library or adaptive retrieval/decision procedures.
Internal actions	retrieve() and LLM-backed propose/evaluate/reflect provide retrieval/reasoning.	Reasoning is split by graph nodes, but currently has no deliberate iterative planning loop.
External grounding	search_catalog, get_history, check_stock; code owns a fixed action whitelist.	Good separation of LLM proposal and executable action space. Tool arguments and business preconditions need typed validation.
Decision procedure	Planning stages followed by execution and feedback.	A one-shot planner inside an iterative action loop; make planning itself iterative and interleavable.
Learning	learn() writes a summary to episodic memory at the end.	This end-of-interaction choice is allowed by CoALA; add validation before memory write and test whether future behavior changes.

CoALA's own retail example describes semantic memory for products, episodic memory for customer purchases/interactions, procedural memory for datastore query functions, and working memory for dialogue state. It

also explicitly says a retail agent may defer learning to the end of an interaction and summarize the episode before storing it. The current implementation is therefore a reasonable small retail mapping; the problem is not deferring learning, but the simplistic and unvalidated record being stored. [R1]

5. Fan & Lan (2026) mechanism audit

Use the paper's seven-field mechanism representation as the review checklist: operative state substrate, control locus, transition trigger, persistence boundary, failure semantics, learning operator, and resource/governance policy. A state field existing in Java is not enough: the mechanism must have an identifiable trigger and authoritative effect. Logging a decision is not the same as controlling a transition, and storing a note is not automatically experience-adapted control. [R2]

Bundle	Paper's proposed coupling	Status in retail-agent.zip	Recommended treatment
B1 — activation / latency / utility	Memory items carry activation, predicted retrieval latency, downstream value, and maintenance cost; context assembly selects under a fixed budget; outcomes update accessibility and utility.	Absent. Retrieval is recent-episode top-3 plus vector top-2. No activation/latency/value/cost fields or downstream-use learning.	Implement after state and workspace semantics are explicit; compare with simpler recency/similarity/top-k baselines.
B2 — typed impasse / substate / compilation	Classify failure, freeze relevant parent state, open bounded recovery subgraph, return typed resolution, compile reusable recovery rule.	Absent. LLM failure counters and deterministic fallback are not a typed impasse substate. Tool errors are strings; no parent snapshot, child state, typed return, or rule compilation.	Build typed failure taxonomy and bounded child recovery first; preserve the intention boundary; compile only verified reusable resolutions.
B3 — content competition / workspace / broadcast learning	Typed memory/agent/tool/monitor/plan candidates compete for a bounded workspace; winner becomes globally available for a cycle; downstream utility updates later admission.	Absent. Prompt concatenates a memory summary and tool observations. Vector top-k is retrieval, not workspace competition/broadcast/admission learning.	Add a bounded typed workspace and explicit admission traces. Compare with top-k; track rare/critical evidence suppression.
B4 — intention / reconsideration / method authority	Persistent intention record plus governor that can continue, delegate, suspend, abandon, or change method; method changes must not silently replace goal.	Absent. Query is a string; reflection can stop/continue, but cannot govern a durable commitment or method switch.	Externalize intention record and explicit reconsideration triggers. Test goal drift, stale commitment, oscillation, correct stop.
B5 — skill governance (closed case)	GraSP composes confidence-based routing between typed skill DAG and ReAct, node pre/postconditions, bounded typed repair, fallback to global replanning/ReAct.	Not present. Tool whitelist is not a typed skill DAG or checked skill execution protocol.	Do not call B5 an unresolved missing bundle. Treat GraSP-style governance as the closest-baseline/control comparator if the project claims skill reuse.
B6 — uncertainty / resources / interruption / stopping	One calibrated uncertainty object propagates through memory admission, planning, tool authorization, monitoring, and stopping, coupled to resource allocation and interruption.	Absent. confidence and score are arbitrary model numbers; threshold only affects first-turn clarification; integer max iterations/failures are static.	Introduce uncertainty with provenance and calibration evidence, propagate it to all decision points, and couple it to explicit token/time/tool/verification budgets.

Important interpretation: B1, B2, B3, B4, and B6 are residual hypotheses in Fan & Lan—not universal stages that every CoALA agent must contain. B5 is closed within the paper's reviewed corpus by GraSP and becomes a baseline for extensions. Fan & Lan explicitly state that the bundles are interdependent and that the proposed synthesis runtime is a test shell, not a mandatory monolith. [R2]

6. Deep review of the simulated LLM stack

6.1 What is well designed

- The provider switch is centralized in LlmStack: scripted, replay, real provider, optional recording, and optional fault decorator are composed in one place.
- ScriptedChatModel is deterministic and fails loudly when no rule matches. That is valuable: an unexpected prompt/task cannot silently receive a fabricated default answer.
- ReplayChatModel fails on a cassette miss, which prevents a missing fixture from silently falling back to live model calls.
- FaultInjectingChatModel is a decorator over any ChatModel, so the same fault layer can wrap a scripted, replay, or real provider.
- The tests isolate at least JSON repair, proposal timeout fallback, low-score clarification, and total provider outage. LlmStackTest checks provider wiring without opening a network connection.
- HashEmbeddingModel provides a deterministic, offline vector for integration smoke tests; it makes CI reproducible without an API key.

6.2 Critical simulator fidelity defects

Component	Observed behavior	Why it weakens evidence	Better design
Scripted proposal	RetailScriptedResponder emits a fixed search → check_stock → finish path based on whether observation tags are present.	Tests encode a happy path rather than testing whether the agent reasons correctly from task facts.	Keep as a fixture, but add adversarial scenario scripts driven by typed state and expected decision traces.
Evaluator	EVALUATE extracts `conf=` values from the proposal prompt and returns those as scores.	This is not an independent evaluation; it simply echoes proposer confidence. It cannot catch a bad or overconfident proposal.	Use a distinct deterministic scoring oracle for tests; test heuristic and model evaluators separately. The test oracle should score constraints/evidence, not proposal confidence.
Reflection	REFLECT returns goalSatisfied=true if the observation contains `[check_stock]`.	A missing product or stock=0 still passes the predicate. The fallback has the same flaw: any stock-check observation ends the run.	Verify structured fields: product exists, constraints match, stock>0, price within budget, and evidence is current. Separate “attempted check” from “successfully verified”.
Answer generation	The script uses a regex over textual stock observations.	It validates only the one pattern; it is not evidence provenance or a general answer policy.	Generate answers from a typed VerifiedProduct/evidence structure; use prose model only as a renderer over trusted facts.
Fault injection	TIMEOUT/MALFORMED/EMPTY/ LOW_CONFIDENCE affect model calls.	Does not inject tool failures, stale inventory, retrieval errors, invalid arguments, cancellation, rate limits, or mid-action partial failures.	Extend to typed failure injection at each boundary, including search, stock, persistence, model and graph node execution.
Low confidence	Regex replaces confidence/scores with 0.1.	Only tests a field manipulation; does not represent calibrated uncertainty or test uncertainty propagation.	Inject explicit uncertainty values plus provenance, and measure decisions at memory, planning, authorization, verification and stopping.
Cassettes	Prompt hash → response text; round-trip test records a ScriptedChatModel, not a real model.	Useful replay plumbing, but does not prove real-model capture/replay. Key omits model/options/schema/version metadata.	Name the test honestly; add one opt-in real capture contract and fixture manifest with model, prompt/schema/tool versions, dependency versions and privacy metadata.

Component	Observed behavior	Why it weakens evidence	Better design
Replay CI	Replay test is skipped unless cassette files exist. Archive has only .gitkeep.	The CI path does not currently prove a committed real cassette can replay end to end.	Commit a small anonymized fixture cassette set or maintain a synthetic scripted cassette test that runs unconditionally.
Cassette privacy/write safety	Cassettes store flattened prompt text including customer identifier and response as plain text; direct Files.writeString.	Potential PII leakage into source control/artifacts; partial writes and concurrent writers are not guarded.	Ignore local real cassettes by default; sanitize IDs; use test-only redacted fixtures; write temp then atomic move; version and integrity-check files.
Embedding substitute	256-dimensional normalized hashed bag-of-words with token collisions, ASCII tokenization and simple plural stripping.	It is deterministic, not semantically meaningful; it cannot validate production semantic retrieval quality.	Call it a deterministic hash baseline. Test retrieval separately using hand-authored deterministic ranking fixtures and a real/local embedding integration suite.

The central problem The default test simulator gives the system the answer to its own evaluation question: the proposer says confidence 0.9, the evaluator returns 0.9, and the selector chooses it. Then reflection reports success because the stock-check tool ran. The happy-path test can pass even when stock is zero or the item violates the user's constraints.

6.3 Tests that pass versus claims they justify

Existing test	What it genuinely verifies	What it does not verify
ScriptedAgentTest	Outer loop, retrieve-after-observation, action whitelist, basic episode reuse in same process.	Planning-time iteration/interleaving, independent evaluator, semantic LLM behavior, durable memory.
FaultInjectionAgentTest	One malformed JSON retry; proposal timeout fallback; low-score branch; global provider-failure termination.	Typed domain failure recovery, tool failure handling, robust stop proof, uncertainty calibration.
CassetteRoundTripTest	A deterministic scripted response can be recorded and replayed byte/text-equivalently.	Real provider capture; version-safe cross-environment replay; tool-use/multimodal replay.
RetailAgentReplayTest	End-to-end replay when cassette fixtures exist.	Runs only when fixtures exist; current archive contains no cassette fixture.
RetailAgentContractTest	Nonempty answer, iteration ceiling, trace endpoints, mentions a catalog ID/name or asks a question.	Evidence-to-answer entailment, verified stock, budget compliance, correct recommendation, calibration.
CatalogSearchTest	The configured local index returns a useful result for a basic fixed query.	Robust semantic recall/ranking across paraphrases, languages, adversarial distractors or real embeddings.

7. A stronger complete retail example

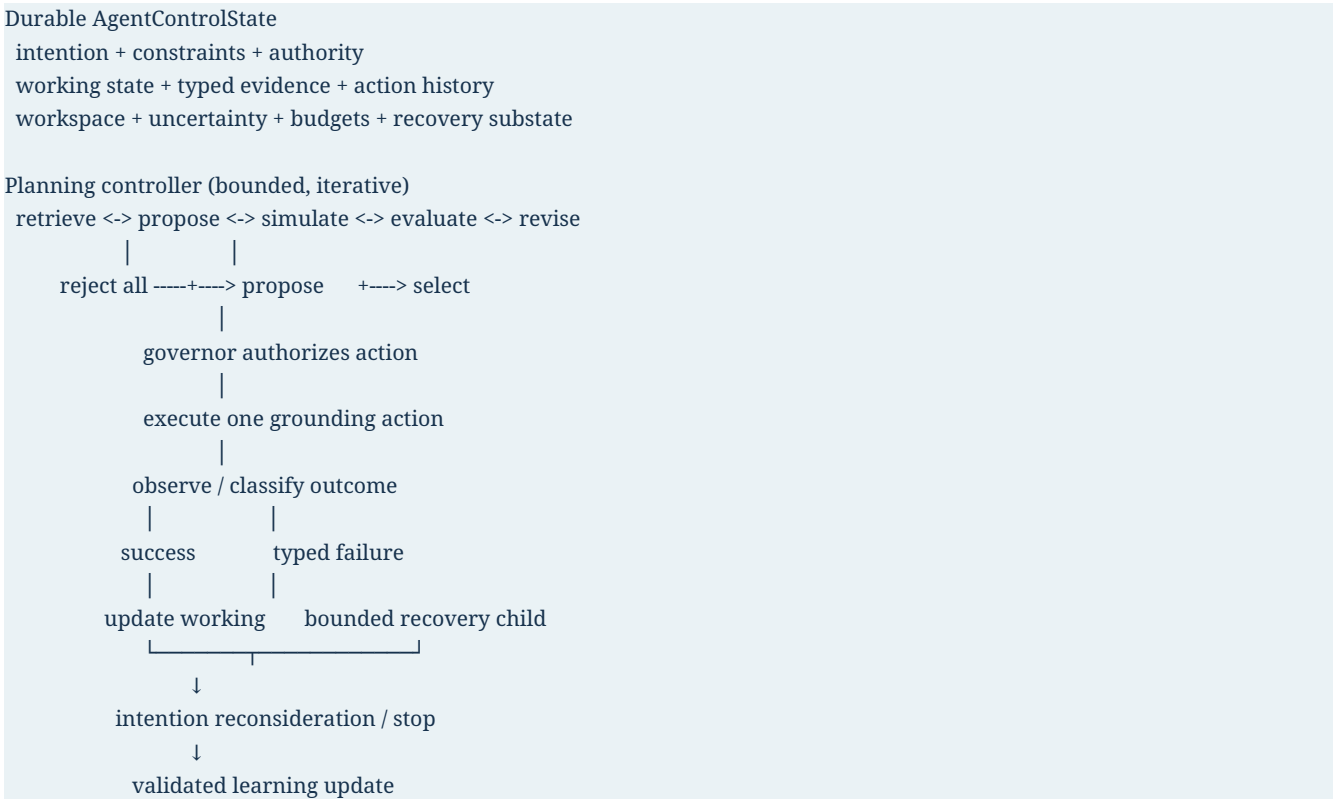
Use a task that forces the agent to revise a candidate rather than marching through a scripted sequence: "Find waterproof trail-running shoes for rocky trails, under $150, and recommend only an item whose stock has been verified." This maps naturally to the uploaded catalog: p-103 is a waterproof trail shoe with a rock plate priced at $149; p-101 is cheaper but its description does not establish waterproofing. Treat current stock as unknown until check_stock returns it.

7.1 One complete run, showing both planning loops

Runtime moment	State / proposal	Control decision
Input / intention	Committed goal = identify a waterproof rocky-trail shoe under $150, currently available. Constraints and customer authorization are typed.	Governor stores goal separately from plan/method.
Context admission	Candidates: query constraints, relevant episodic preference, catalog facts for p-101/p-103, current evidence. Every item has type, source, freshness and estimated token cost.	B3-like bounded admission selects evidence; omitted evidence and reasons are logged.
Planning pass 1	Candidate A: recommend p-103 immediately. Candidate B: search for waterproof trail alternatives. Candidate C: inspect prior purchase history.	Evaluation finds A unsupported: no stock evidence. Selection rejects A; it does not force execution.
Planning iteration	Reasoning detects missing availability evidence. Retrieval narrows catalog and history. The proposal is revised: search/verify p-103 and alternatives rather than recommend now.	This is the CoALA selection→proposal loop and retrieval/reasoning interleaving.
Grounding action	Execute only the authorized search action. Return typed SearchResult records with catalog IDs and source.	External action is executed after code-side schema, permission and precondition checks.
New decision cycle	Observation includes p-103 and alternatives. Re-admit context; propose stock checks for candidate IDs; evaluate against constraints and remaining budget.	A selected read-only stock check executes. The cycle repeats with fresh evidence.
Typed failure branch	Suppose p-103 returns stock=0. Record `NO_STOCK` with tool, item, timestamp, evidence and parent state hash.	B2 opens a bounded child recovery substate. The parent intention is preserved; child may search alternatives.
Method reconsideration	A diagnosis says “candidate unavailable”; choose a different search strategy or broaden terms while keeping waterproof / rocky trail / under $150.	B4 changes method, not the committed goal. A human escalation is possible if no safe candidate remains.
Uncertainty/ resource gate	Stock evidence is stale or conflicting: uncertainty carries provenance; remaining calls/tokens/time are bounded.	B6 allocates verification or stops/asks for help based on risk and remaining budget.
Final selection	A candidate is eligible only if all hard constraints and inventory evidence pass. A proposal cannot self-authorize stock or price facts.	The governor authorizes the final action or refuses to recommend.
Learning	Store a validated episode: constraint profile, selected candidate, evidence IDs, failure/recovery type and outcome.	B1/B3 updates only from observed downstream use; B2 compiles only a verified reusable recovery; B6 calibration is updated from outcomes.

This is an illustrative engineering design, not a prescribed linear sequence from either paper. The graph should branch and loop according to conditions; a node list is only the implementation surface. CoALA supports this flexibility in planning. Fan & Lan explicitly describe the B4→B2 commitment/recovery dependency, B1 and skills supplying B3 candidates, B3 evidence triggering B4 reconsideration, and B6 governing verification, authorization, interruption and stopping. [R1][R2]

8. Target design: one conceptual core, explicit control authority



Core object / contract	Recommended fields (illustrative)	Why it must be explicit
IntentionRecord	goalId, committedGoal, constraints, rationale, planRef, continuationCondition, reconsiderationTriggers, authority, status, version	Goal persistence and method-change authority; prevents silent goal drift.
ActionCandidate	candidateId, actionType, typedArgs, rationale, confidence/uncertainty, preconditions, predictedEffects, provenance	Separates model proposal from trusted argument validation and executable authority.
EvidenceItem	evidenceId, type, source, payloadRef, observedAt, freshness, trust/provenance, relevance, tokenCost	Separates raw reports, tool observations, inferred beliefs and verified facts.
UncertaintyRecord	estimate, calibratedClass/interval, provenance, stage, calibrationVersion, consequence	One uncertainty object can propagate to retrieval, evaluation, tool authorization, verification and stop.
BudgetState	deadline, model/token budget, tool-call budget, verification budget, retry budget, cancellation state	Static maxIterations is not a complete resource/governance policy.
FailureRecord / RecoveryState	failureType, failingAction, parentStateHash, bounded child state, typed resolution, return policy	Makes B2 observable and protects parent intention/state during recovery.
WorkspaceSnapshot	candidate IDs, scores, winner, recipients, admission reason, capacity, downstream use	Proves competition, scarcity, broadcast and outcome-adapted admission rather than simple message passing.

Keep authoritative state outside the LLM prompt. The model can propose and interpret; a deterministic governor validates and authorizes state transitions. This is the control boundary in Fan & Lan's bounded synthesis runtime. [R2]

9. Recommended implementation sequence

Phase	Implementation work	Acceptance evidence
0 — stabilize current project	Add Serializable to Proposal; add serialization contract test. Fix misleading AgentSettings comment about default step limits; configure graph execution bounds explicitly when supported. Replace process-local ArrayList episodic data with thread-safe snapshots or persistence.	Compilation and existing tests remain green; serialization test round-trips all current state shapes; concurrent memory test is deterministic.
1 — make CoALA planning correct	Add explicit planningRound and planning outcome state. Add reject-all → proposal edge. Allow retrieve/reason/evaluate/revise to interleave. Add bounded candidate rollouts/simulation as an optional procedure; preserve select as authority.	Tests show at least two planning rounds before one grounding action; retrieval can be invoked between proposal and evaluation; rejected candidates never execute.
2 — authoritative state + governance	Persist intention, budgets, uncertainty, recovery status and action history outside prompts. Give every run stable conversation/thread ID and explicit checkpoint/resume semantics.	Crash/restart resumes same intention and node position; state ownership and transitions are traceable; prompt content is not the authority source.
3 — typed failure / bounded recovery	Classify tool/model/retrieval failures; freeze relevant parent state; enter bounded recovery subgraph; return typed resolution. No raw error strings masquerade as observations.	Fault matrix covers NO_STOCK, INVALID_ARGUMENT, NOT_FOUND, STALE_DATA, TIMEOUT, RATE_LIMIT, PROVIDER_DOWN, RETRIEVAL_FAILURE; parent hash and typed return are asserted.
4 — bounded workspace admission	Represent candidate evidence types and token cost; score/compete under a capacity; broadcast winner to configured recipients; record downstream use and admission updates.	Compare against top-k baseline; measure evidence density, distractor robustness, context cost and rare-critical-evidence suppression.
5 — B4/B6 governance	Add explicit intention lifecycle and method switching; introduce calibrated uncertainty provenance and a shared resource scheduler for tool authorization, verification, interruption and stopping.	Measure goal drift, stale commitment, oscillation, selective risk, calibration, false stop, interruption latency and irreversible-action errors.
6 — adaptive memory / skill policies	Implement B1 activation-latency-value-cost only against a baseline. Add a GraSP-like typed skill-DAG baseline before compiling B2 resolutions into skills.	Ablations show value beyond recency/top-k/simple retry; unsafe or inapplicable recovery rules never compile or run.

The Fan & Lan paper's own implementation-order recommendation is to externalize intentions, uncertainty, budgets and recovery state first; then implement typed failure and bounded content admission; then adaptive memory and compiled skill policies. It warns that automatic skill compilation before state and failure boundaries are explicit can produce unauditable shortcuts. Treat this as the paper's recommended order for its proposed synthesis runtime—not as a universal requirement for every CoALA agent. [R2]

10. Testing and evaluation: what would count as evidence

10.1 Replace "test the happy path" with adversarial scenario fixtures

- No results; ambiguous user constraint; history conflicts with current request; best semantic match violates a hard constraint.
- First candidate out of stock; second candidate in stock but over budget; all candidates out of stock; stock observation stale or contradictory.
- Malformed proposal; valid JSON but wrong schema; unsupported action; valid tool with invalid args; evaluator disagrees with proposer; all candidates rejected.
- Model timeout during proposal/evaluation/reflection/answer; tool timeout after partial completion; rate limit; retrieval/vector store down; checkpoint write fails.

- B2: correct and incorrect failure diagnosis, recovery child exceeding budget, parent-state corruption attempt, unsafe compilation attempt, repeated identical failure.
- B3: rare security/constraint evidence competing against many high-similarity distractors; compare workspace to top-k.
- B4: tempting method switch that changes goal; stale intention; alternating method changes (oscillation); legitimate suspend/resume.
- B6: overconfident incorrect stock claim, calibrated low confidence, high risk with low remaining budget, interruption before irreversible tool action.

10.2 Required assertions

Invariant	Test should assert
Proposal is not authority	Unknown tool / invalid args / failed precondition never executes, even with confidence=1.0.
Planning rejection is real	If all proposals are rejected, control returns to proposal or safely terminates; no tool executes.
Simulation is not execution	Hypothetical rollouts cannot mutate inventory, order history or real tools.
Truthful success condition	A stock-check invocation alone is not success; product identity, stock, price and constraints must pass.
Typed failure boundary	Error kind, parent snapshot/hash, child trace and resolution are persisted and auditable.
Goal/method separation	A method switch never silently changes constraints or committed goal.
Uncertainty propagation	The same uncertainty provenance is traceable at retrieval, planning, authorization, verification and stopping.
Learning is gated	Only validated outcomes update durable memory/skill/admission policies; learning effects appear in later decision traces.
Resource governance	Planning loops and recovery stop at explicit time/token/tool/retry budgets; interruption is tested.

Fan & Lan require matched model, prompt, tools, environment and task budget when evaluating a proposed bundle, plus instrumentation of internal transitions. The paper lists three useful ablations: remove the new state object, remove the authoritative control edge while retaining logging, and remove the learning update while retaining the runtime transition. It also defines bundle-specific rejection conditions. [R2]

11. Is a later LangChain4j implementation a good idea?

Recommendation Yes—build a second implementation to compare orchestration semantics, but do not copy the current Spring-AI/LangGraph4j classes wholesale. Reuse a framework-neutral agent core and supply two framework adapters.

The current project pins LangGraph4j 1.8.25 and Spring AI 2.0.1. As of 4 November 2026, the LangGraph4j release page lists 1.9.8 (released 3 November 2026), while LangChain4j's latest listed release is 1.22.0 (released 8 October 2026). Pin exact versions for both experiments and upgrade the current project separately. [R3][R5]

Concern	LangGraph4j version	LangChain4j version	Portability implication
Main abstraction	Explicit StateGraph/CompiledGraph nodes, edges, conditional transitions and graph state.	Agentic abstractions include agents, workflows, planners, AgenticScope, tool agents and custom Planner implementations.	The graph is not a direct one-to-one mapping to an AgenticServices sequence. Preserve semantics in your own controller.
Control-flow freedom	Natural fit for explicit loops, branches, child recovery subgraphs and visible transition edges.	Built-in sequence/parallel/loop/supervisor patterns plus custom Planner API.	A custom Planner is likely needed to reproduce the same proposal/evaluate/reject/revise/govern loop.

Concern	LangGraph4j version	LangChain4j version	Portability implication
Persistence	Graph state and checkpoint savers are first-class concepts in the LangGraph4j ecosystem.	AgenticScopeStore SPI; per-step scope checkpointing and planner execution-state persistence are documented.	Both can persist, but the persisted schema, serializers and resume semantics are framework-specific.
Custom state types	Your current version relies on serializable state-copy behavior; Proposal is the immediate failure.	AgenticScope JSON serialization uses an allowlist; custom domain classes must be explicitly registered before deserialization.	Use simple, versioned DTOs and test both serializers; do not assume Java Serializable solves LangChain4j persistence.
Human approval	Can be expressed as graph interrupt/wait/resume, depending on selected modules and saver.	Agentic module provides HumanInTheLoop and suspended/resumable workflow support.	A good portability comparison, but keep the business approval state independent of either runtime.
Maturity / change risk	LangGraph4j current 1.9 line is evolving; release notes show checkpoint, resume, saver and hook changes.	Official docs label `langchain4j-agentic` experimental and subject to change.	Pin and isolate framework adapters; do not place experimental framework APIs in the core domain model.

LangChain4j's official documentation supports custom planners and persistence: its Planner can save and restore internal execution state; when AgenticScopeStore is configured, the scope is checkpointed after agent invocations and the workflow can resume from the saved planner position. Its AgenticScope JSON deserializer restricts custom classes through an allowlist. The same docs explicitly mark the `langchain4j-agentic` module experimental. [R4]

11.1 The architecture to share

```
retail-agent-core/
  domain/         Intention, Evidence, Proposal, Plan, Failure, Budget
  memory/         EpisodicMemory, SemanticCatalog, ProceduralSkills
  control/        PlanningController, Governor, RecoveryPolicy, StopPolicy
  ports/          ModelPort, RetrievalPort, ToolPort, StateStore, Clock
  evaluation/     DeterministicOracle, CalibrationTracker, TraceSink

retail-agent-langgraph4j/   StateGraph adapter + checkpoint adapter
retail-agent-langchain4j/   Custom Planner / AgenticScope adapter
retail-agent-spring/        Spring Boot configuration + provider adapters
```

The core should not import `org.bsc.langgraph4j.*`, `org.springframework.ai.*`, or `dev.langchain4j.*`. For the current project, define an app-level `ModelPort` and tool contracts first; wrap Spring AI ChatModel behind them. In the second implementation, adapt LangChain4j ChatModel/AI Services to the same core. Keep the state DTOs framework-neutral and JSON-friendly.

11.2 What should be shared versus reimplemented

Share exactly	Implement separately
Domain state and its invariants; action schemas; tool authorization; typed evidence and failure taxonomy; intention lifecycle; budgets; uncertainty provenance; workspace admission policy; learning rules; simulator scenarios; evaluation datasets and metric formulas.	LangGraph4j node/edge wiring, checkpoint serializer/saver and graph-level interrupt/resume adapter.
Prompt templates or structured model request contracts where framework-neutral; deterministic oracle; cassette manifest format; audit event schema.	Spring AI and LangChain4j provider wrappers, structured-output bindings, model options, retries and token-usage extraction.
Expected traces and contract tests that run against a runtime-neutral controller.	Custom LangChain4j Planner, AgenticScope state serialization allowlist, store SPI and suspended response handling.

Do not compare “LangGraph4j versus LangChain4j” using different prompts, models, tools, budgets, embeddings or data. Reuse the same test scenarios and evaluator. Otherwise differences in outcomes cannot be attributed to the orchestration framework. This mirrors the controlled-comparison discipline Fan & Lan request. [R2]

12. Priority decisions and what not to claim

Priority	Decision
Do now	Apply `Proposal implements Serializable`; add a serializer regression test; keep the current pinned dependency while confirming the baseline build.
Do next	Refactor the planner so rejection returns to proposal and retrieval/reasoning/evaluation can interleave under a hard planning budget.
Do before adaptive learning	Externalize authoritative intention, uncertainty, budgets and recovery state; introduce typed evidence and typed failures.
Then	Build bounded recovery (B2) and bounded workspace admission (B3), with B5/GraSP treated as a baseline; later add adaptive memory (B1), B4 commitment/method governance and B6 uncertainty/resource coupling.
For tests	Keep scripted LLM for deterministic flow tests, but stop treating it as an LLM-quality simulator. Introduce an independent rule oracle and adversarial fixtures; distinguish scripted cassette tests from real-provider cassette capture.
For portability	Extract a small framework-neutral core; build a second LangChain4j adapter around a custom Planner rather than translating StateGraph nodes into a rigid sequence.

Avoid these claims until the corresponding tests and traces exist:

- “The planner is iterative” — only the outer action cycle currently repeats; the planning sub-process is one-pass.
- “The evaluator independently evaluates candidates” — the scripted evaluator currently copies proposer confidence.
- “The agent verifies stock” — it calls the stock tool, but the scripted reflection regards any check as success, including unknown/out-of-stock observations.
- “The simulator mimics an LLM” — it is a deterministic scripted ChatModel test double with a good fault/replay harness.
- “B1/B2/B3/B4/B6 are implemented” — their required state/control/persistence/learning couplings are absent.
- “B5 is missing research” — B5 is closed by GraSP in Fan & Lan’s reviewed corpus; compare against that protocol rather than calling it an unresolved bundle.
- “LangChain4j is a drop-in replacement” — the agentic layer is experimental and uses a different planning/state/persistence model.

13. Sources and review boundaries

Primary papers

- [R1] Sumers, T. R., Yao, S., Narasimhan, K. R., & Griffiths, T. L. (2024). Cognitive Architectures for Language Agents. Transactions on Machine Learning Research. arXiv:2309.02427v3. Particularly §§4.1–4.6 and §6. https://arxiv.org/pdf/2309.02427v3
- [R2] Fan, H., & Lan, Z. (2026). From Cognitive Architectures to Language Agents: A Mechanism-Level Review of Lineage, Convergence, and Migration Gaps. arXiv:2607.23942v1. Particularly §§II–III, V, VI-A–VI-E, VII, Appendices C–D. https://arxiv.org/html/2607.23942

Framework documentation checked on 4 November 2026

- [R3] LangGraph4j GitHub releases. The release list identifies v1.9.8 as released on 3 November 2026. https://github.com/langgraph4j/langgraph4j/releases
- [R4] LangChain4j, Agents and Agentic AI documentation. Covers custom Planner, Human-in-the-Loop, AgenticScopeStore, execution-state persistence, JSON deserialization allowlisting; labels the agentic module experimental. https://docs.langchain4j.dev/tutorials/agents/
- [R5] LangChain4j GitHub releases. The current release page lists 1.22.0, released on 8 October 2026. https://github.com/langchain4j/langchain4j/releases

Review boundary: source code and tests inside the supplied retail-agent.zip were inspected. The report uses the user's statement that the build compiles after the Proposal serialization fix and that the listed tests pass; Maven was not rerun during this review. The prior user-generated CoALA/Fan & Lan paper-grounded report was also checked for terminology and scope consistency. Framework release facts were checked against the official project documentation/release pages on the review date.

Bottom line Keep the deterministic test harness, but make it honest: it should prove graph wiring, invariants and failure handling—not pretend to grade reasoning. First make CoALA planning iterative inside each cycle. Then externalize the control state and add the Fan & Lan couplings in the paper's dependency-aware order. A framework-neutral core will let the same agent semantics be implemented and compared on LangGraph4j and LangChain4j.